



Instituto Tecnológico de Hermosillo

Ing. En Sistemas Computacionales

Materia: Programación Web

Grupo: S6A

Nombre del Proyecto: UniMed Control

Integrante: Sebastián Nuño Melendrez

Profesor: Jonathan Adrián Moroyoqui Ibarra

Lunes 01 de Junio de 2026

INTRODUCCIÓN.

UniMed simula una web de administración diseñada para optimizar el control de registros médicos internos en un hospital. La aplicación ayuda a la integración entre las distintas áreas de una clínica al digitalizar y centralizar los catálogos de pacientes, personal médico y la asignación de citas en tiempo real.

Su objetivo general es el de proveer una interfaz rápida y practica que permita hipotéticamente al personal administrativo de un hospital realizar operaciones CRUD, manteniendo la integridad de los datos de las consultas mediante una arquitectura desacoplada.

DOCUMENTACIÓN TÉCNICA.

Tecnologías utilizadas

- **Back End:** ASP.NET Core Web API (.NET 10)
- **Librerías NuGet:** Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Design, Swashbuckle.AspNetCore (Swagger).
- **Front End:** React (Vite)
- **Librerías npm:** react-router-dom, axios
- **Base de Datos:** SQL Server

Arquitectura

El sistema utiliza una arquitectura cliente-servidor desacoplada. El Back End actúa como una API RESTful encargada de la lógica de negocio, validación de datos y persistencia en la base de datos a través de Entity Framework Core. El Front End en React funciona de manera independiente, consumiendo los endpoints de la API mediante peticiones HTTP asíncronas con Axios. La comunicación entre ambos entornos se habilita de forma segura mediante una política CORS configurada en el servidor.

Base de Datos

- **Paciente:** Almacena los datos personales del usuario. Atributos: Id (Key), Nombre, Telefono, FechaNacimiento.
- **Medico:** Almacena la información del personal de salud. Atributos: Id (Key), Nombre, Especialidad, Telefono.
- **CitaMedica:** Entidad intermedia que gestiona las consultas. Atributos: Id (Key), FechaHora, Diagnostico, Tratamiento, PacienteId (FK), MedicoId (FK).

Relaciones: Un paciente puede tener múltiples citas médicas. Un médico puede atender múltiples citas médicas. Las relaciones están protegidas mediante restricciones de clave foránea.

API Endpoints

- GET /api/médicos: Obtener todos los médicos registrados.
- GET /api/medicos/{id}: Obtener los detalles de un médico específico.
- POST /api/médicos: Registrar un nuevo médico.
- PUT /api/medicos/{id}: Actualizar los datos de un médico existente.
- DELETE /api/medicos/{id} : Eliminar un médico del sistema.
- GET /api/pacientes: Obtener todos los pacientes registrados.
- GET /api/pacientes/{id}: Obtener los detalles de un paciente específico.
- POST /api/pacientes: Registrar un nuevo paciente.
- PUT /api/pacientes/{id}: Actualizar los datos de un paciente existente.
- DELETE /api/pacientes/{id}: Eliminar un paciente del sistema.
- GET /api/citasmedicas: Obtener el listado general de citas.
- POST /api/citasmedicas: Agendar una nueva cita.

DISTRIBUCION DEL TRABAJO.

- Commit 1: Agregar gitignore
- Commit 2: agregar archivo de proyecto
- Commit 3: agregar configuracion de app
- Commit 4: configurar programa principal
- Commit 5: agregar modelos de datos
- Commit 6: agregar objetos de transferencia
- Commit 7: contexto de base de datos
- Commit 8: agregar fabrica de contexto
- Commit 9: agregar migraciones
- Commit 10: controlador paciente
- Commit 11: controlador medico
- Commit 12: controlador cita medica
- Commit 13: componentes frontend
- Commit 14: paginas interfaz
- Commit 15: configuracion frontend

RETROALIMENTACIÓN DEL PROYECTO

Sebastián Nuño Melendrez

- **Lo más difícil y solución:**

Lo más complicado fue solucionar los problemas de comunicación inicial entre React y la API, específicamente con el manejo de rutas. Lo resolví configurando de manera estricta las políticas en el archivo Program.cs del backend y alineando las rutas exactas en el frontend.

- **Herramientas consultadas:**

Documentación oficial de Microsoft Learn para Entity Framework Core y la documentación de React Router para la navegación.

- **Qué cambiaría con más tiempo:**

Implementaría un sistema completo de autenticación y roles de usuario para restringir el acceso a ciertos módulos administrativos, además de agregar mas campos de información tanto sobre los pacientes como los médicos, y detalles para programar las citas medicas.

- **Aprendizaje clave:**

Al realizar tan a profundidad todas estas instrucciones, entendí cómo es estructurar una arquitectura limpia comunicando dos entornos independientes desde la interfaz hasta la persistencia en la base de datos. Fue muy útil sobre todo por que no había hecho uso de github de esta manera para almacenar los commits de todo el proceso.

RETROALIMENTACIÓN DEL CURSO

- **Temas más útiles:**

La explicación práctica de la arquitectura Web API y cómo estructurar los controladores para exponer servicios, además de usar los commits en github.

- **Temas que requería más práctica:**

La configuración detallada de relaciones complejas tanto con el Backend como el Frontend.

- **Agregar o quitar:**

Añadiría una pequeña sección enfocada en el despliegue práctico de aplicaciones en la nube.

- **Calificación:**

9/10. El contenido es muy completo y aplicable a proyectos reales, ya que realizamos por la mayoría del curso prácticas, por lo que la transición entre back y front se entiende bien.

CONCLUSIONES.

Este proyecto me permitió consolidar los conocimientos adquiridos a lo largo del curso mediante un escenario práctico real. Se cumplió con el objetivo planteado al inicio del proyecto, entregando una plataforma funcional que integra de manera correcta el almacenamiento de datos relacionales con una interfaz de usuario dinámica y responsiva. Dividir las capas de la aplicación con claridad, facilitando la integración del código mediante el flujo de trabajo establecido en Git fue un proceso muy complejo pero importante.